Week 12
LLMs and RAG

Hwanjun Song
KAIST, Dept. of ISE

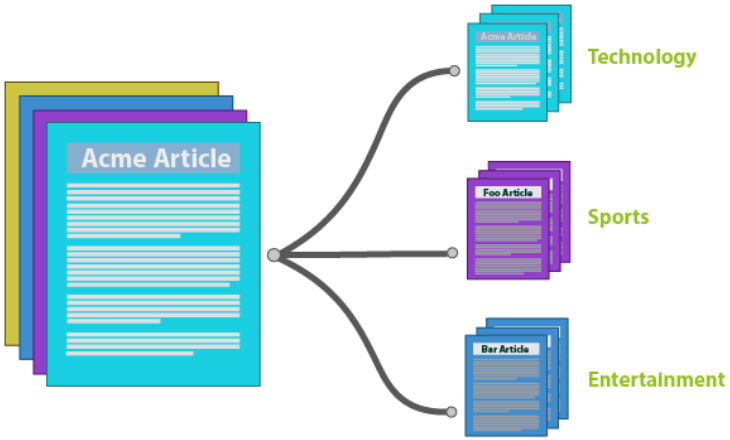
Today's Contents

- Discriminative vs. Generative LMs <<
- Large Language Models
  - Pre-training with Next Token Prediction
  - Instruction Fine-tuning
  - Preference Optimization
  - Prompt Engineering
- Retrieval Augmented Generation
  - Chunking, Embedding, Vector Database, Retrieval, Generation
  - Evaluation of the Retrieval: NDCG

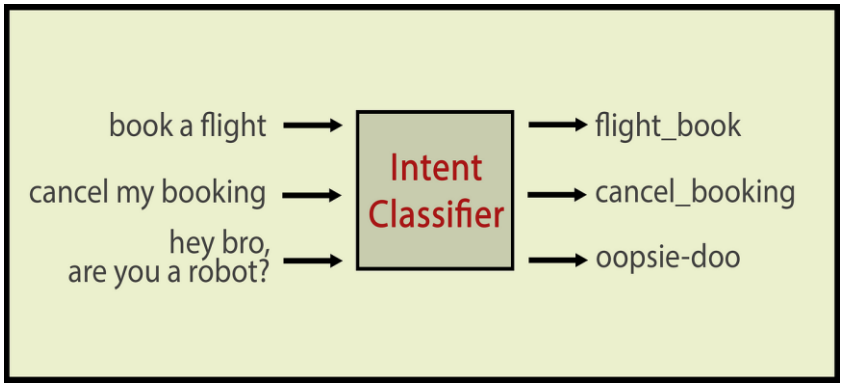
This is an introductory class, so we will not dive into the detailed of each topic.

Discriminative Tasks in NLP

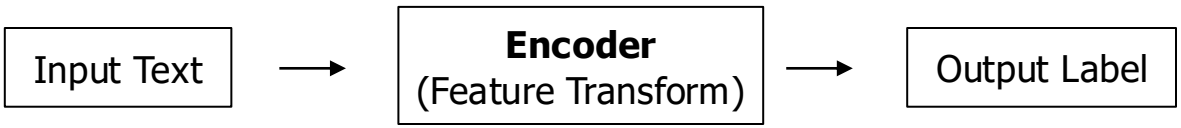
- A task that returns a specific type of output we pre-defined, such as a category (classification) and numerical value (regression)
  - E.g. 1, Text Classification



  - E.g. 2, Intent Classification

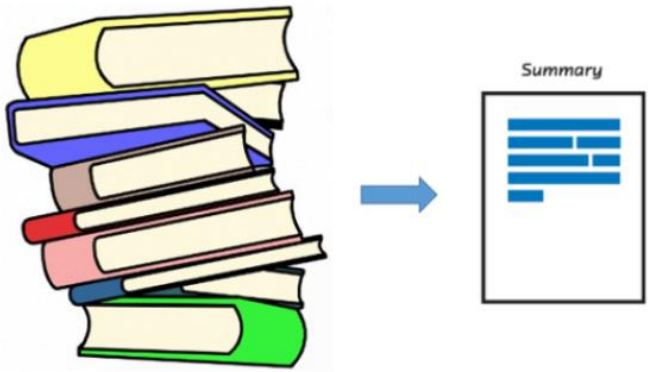


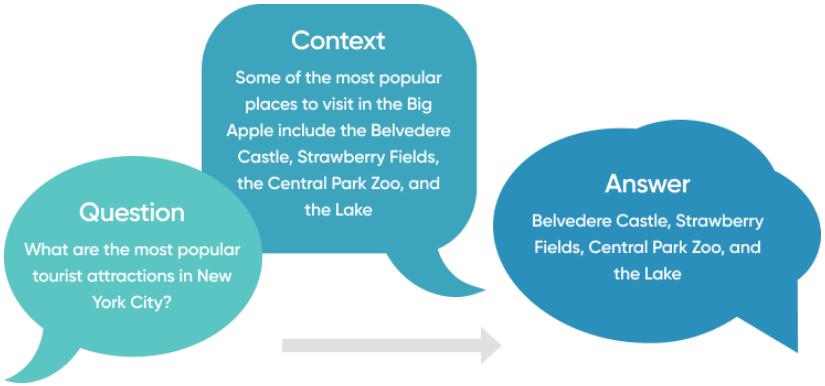
- Inference by Discriminative Language Models (Encoder model)
  - Output by the model is **simple**
  - A **single** forward propagation is enough to get output (probs, numbers)
  - In NLP,



Generative Tasks in NLP

- A task that generates new content in text format
- E.g., Think what we do using ChatGPT
  - E.g. 1, Summarization
  - E.g. 2, Question Answering



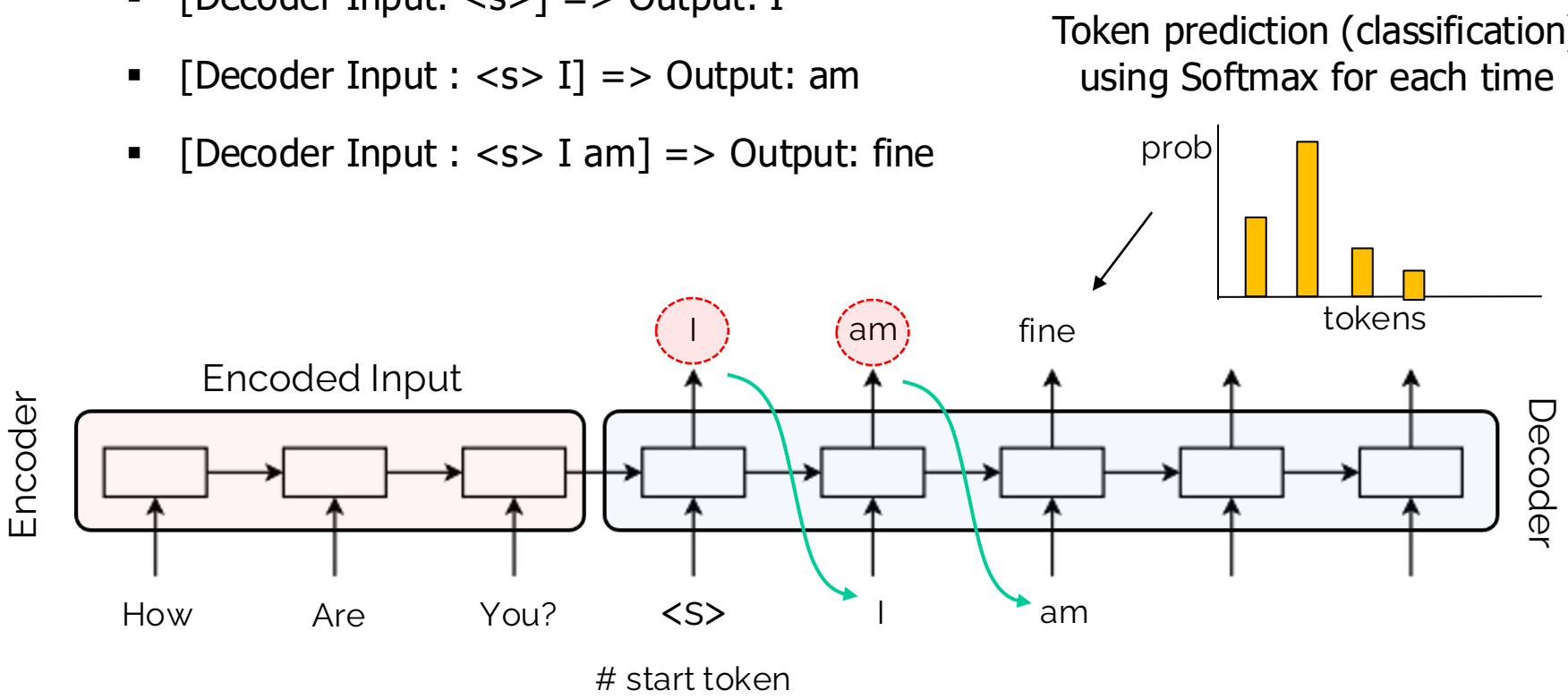


- Inference by Generative Language Models (Encoder-Decoder, Decoder-only)
  - **Multiple** forward propagation is required to get output text
  - Output Layers in Models is also fully connected layer (token classification)



Decoder (e.g., RNN Decoder, Transformer Decoder)

- A neural architecture designed for **sequence generation**, predicting each next token sequentially based on all input tokens and generated tokens so far.
- So… **decoding** is a sequence of multiple token prediction tasks
  - [Decoder Input: <s>] => Output: I
  - [Decoder Input : <s> I] => Output: am
  - [Decoder Input : <s> I am] => Output: fine



Formally, but too costly

- Given input text (docs, user prompt, etc): D , e.g., How are you?
  - At t=1, $P(\hat{y}_1|\hat{y}_{<t=1}, D)$
  - At t=2, $P(\hat{y}_2|\hat{y}_{<t=2}, D)$
  - …
  - At t=T, $P(\hat{y}_T|\hat{y}_{<t=T}, D)$

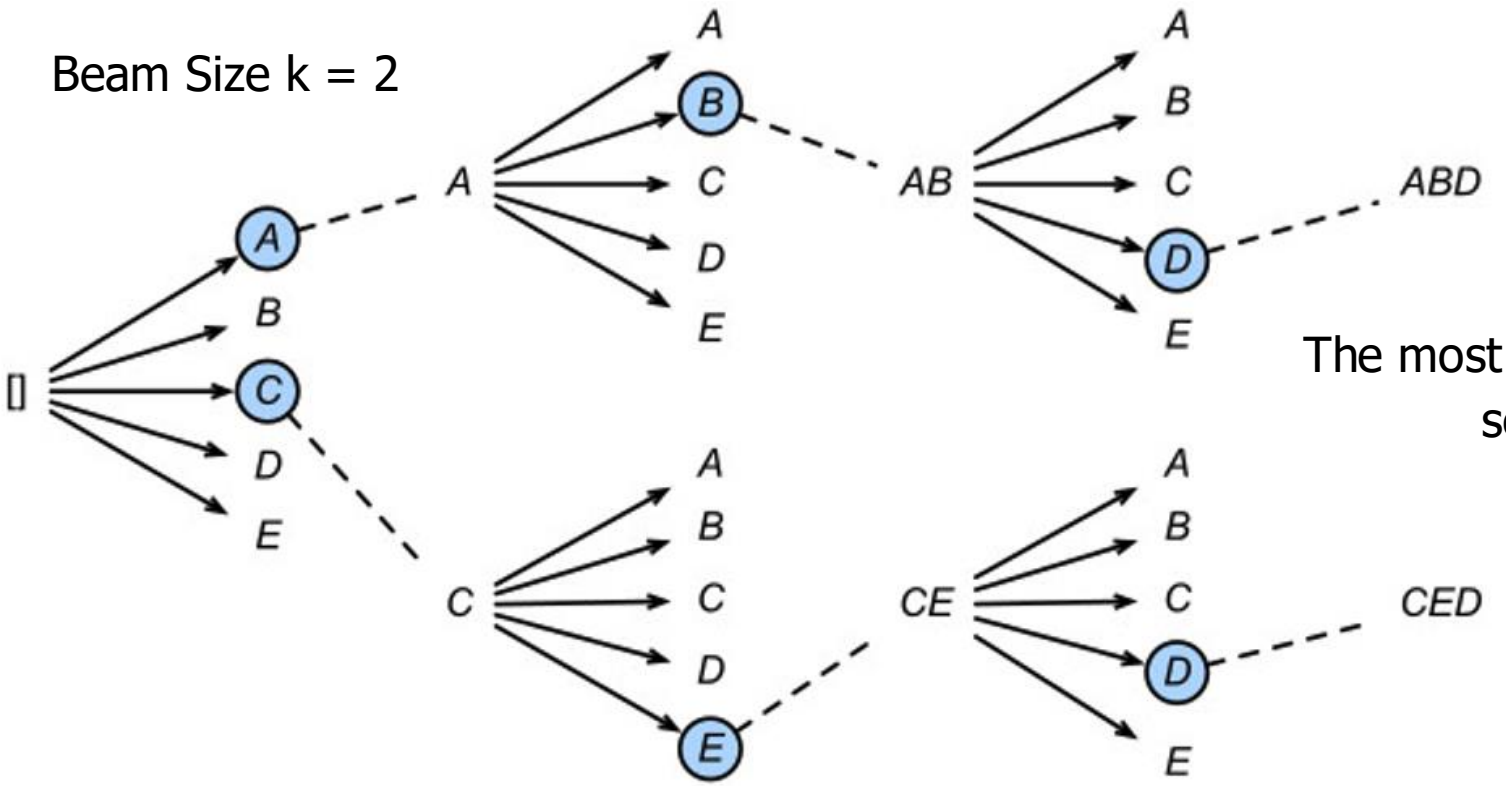
$\hat{y}_{<t}$: all the generated (predicted) tokens before t
$\hat{}$: predicted token (not ground truth)

- What is the generated sequence S?
  - $P(\hat{S} = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T\} \mid D) = \prod_{t=1}^{T} P(\hat{y}_t|\hat{y}_{<t}, D)$
    - The joint probability to generate the sequence $\hat{S}$
    - e.g., a candidate for $\hat{S} = \{I, am, fine\}$
  - We need to find the **most probable** sequence S* as the final response
    - $S^* = \text{argmax}_{\hat{S}} \; P(\hat{S} \mid D)$
    - But, this is NP-hard problem as the space of possible sequences $\hat{S}$ grows exponentially with the length of the sequence T

KAIST

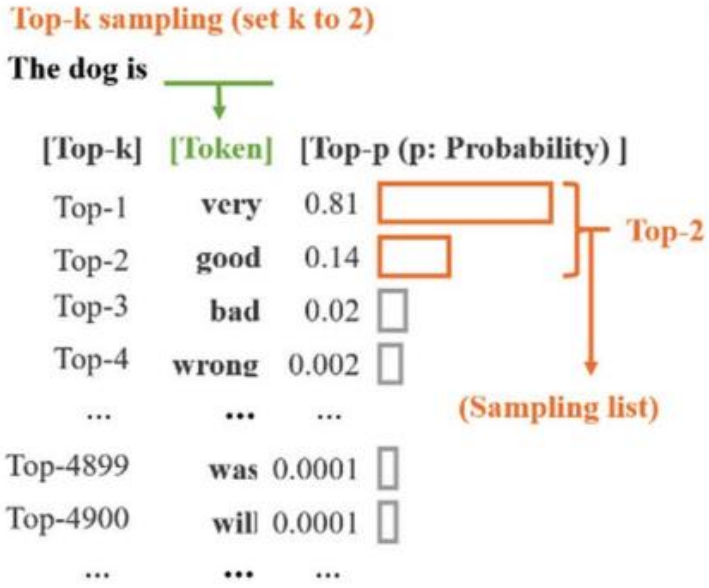
Approximated Search Methods: Deterministic Search

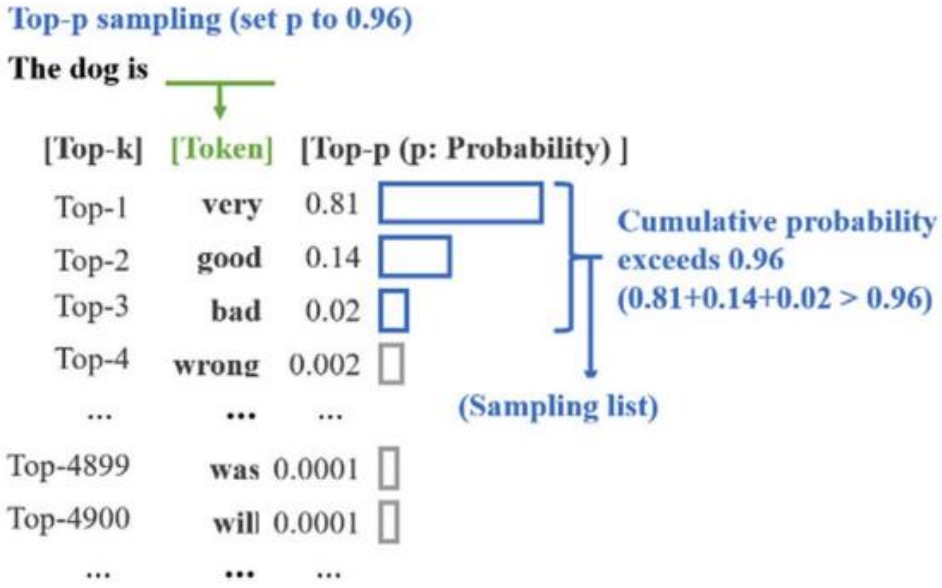
- The generated (selected) sequence is the same as long as the input is the same.
- **Beam Search**: a method to keep only the top k sequences (beams) with the highest probabilities at each step (better than greedy)



Approximated Search Methods: Non-Deterministic Search

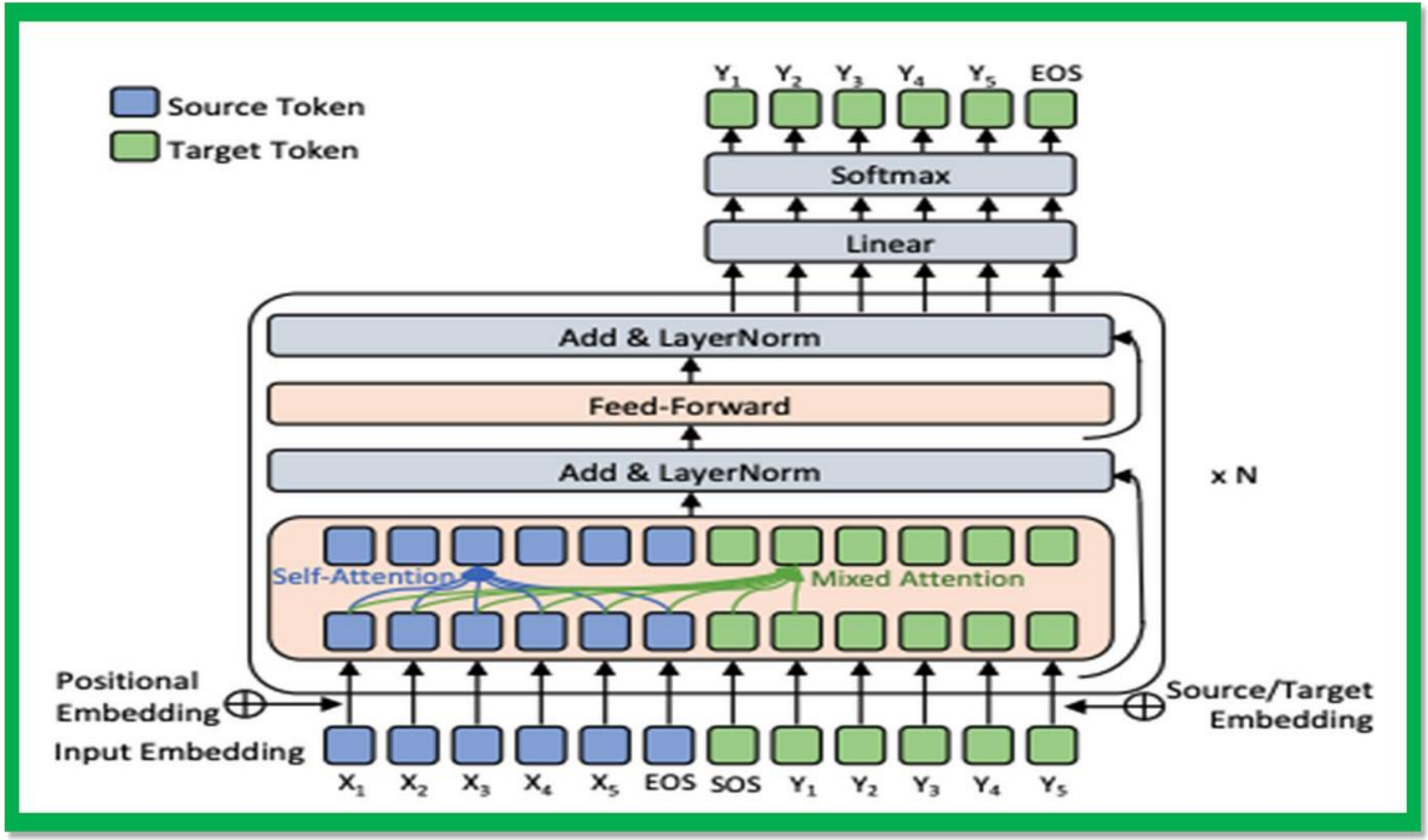
- We can add some **sampling techniques** into the search pipeline, **increasing diversity** of the generated output, like ChatGPT
- Top-k Sampling
  - For each step, samples a token from tokens with k largest probs; $P(\hat{y}_t|\hat{y}_{<t}, D)$
- Top-p (nucleus) Sampling
  - For each step, samples a token from (sorted) tokens up to the moment when their cumulative prob becomes greater than threshold p





Move towards "Decoder-only" Model

- Aggregates the Encoder (encoding) into Decoder with Masking



Summary

KAIST

Today's Contents

- Discriminative vs. Generative LMs
- Large Language Models <<
  - Pre-training with Next Token Prediction
  - Instruction Fine-tuning
  - Preference Optimization
  - Prompt Engineering
- Retrieval Augmented Generation
  - Chunking, Embedding, Vector Database, Retrieval, Generation
  - Evaluation of the Retrieval: NDCG

What Makes LLMs Different from LMs?

- Common Point: Both Models aim to generate better responses given user input – mostly for generative tasks.
- Difference 1. Transition to Decoder-only model
  - Simple architecture by removing the Transformer encoder
  - More efficient for text generation by doing encoding together with decoding

Encoder-Decoder Model

- BART, Facebook AI, 2019
- T5, Google Research, 2020
- Long Former, AI2, 2021
- Flan-T5, Google Research, 2022

Since 2023, decoder-only becomes default

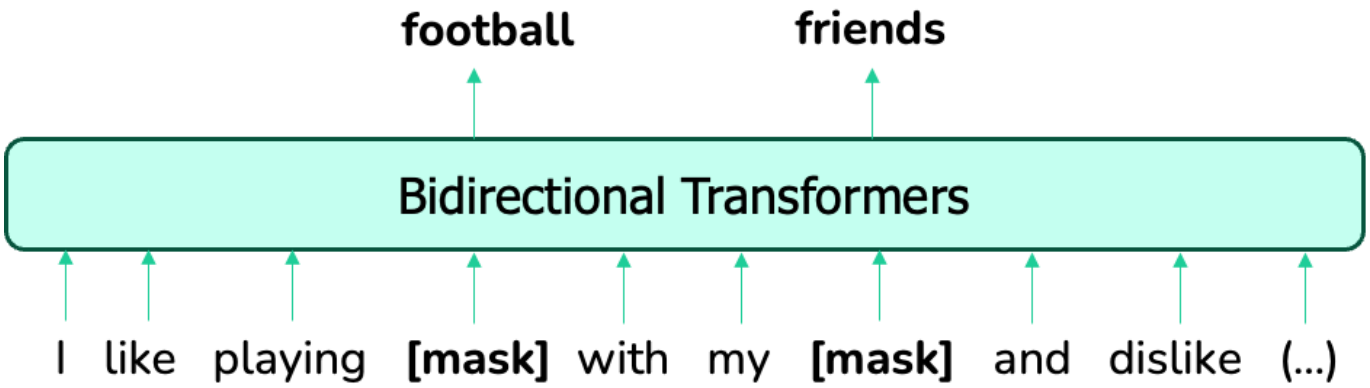
Because improvement of
(1) memory usage;
(2) inference speed

- ChatGPT, OpenAI, 2023
- Phi-2, Microsoft, 2023
- Llama2, Meta AI, 2023
- Mixtral, 2023
- Claude2/3, Anthropic, 2023
- GPT-4 OpenAI, 2024
- Llama3, MetaAI, 2024, etc

Decoder-Only Model

What Makes LLMs Different from LMs?

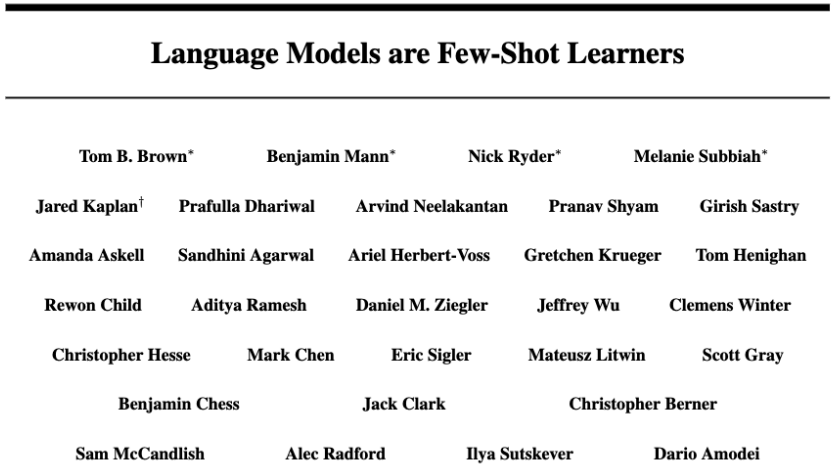
- Difference 2. Large-scale **Pre-training** for Text Vectorization
  - **Previously**: mostly BERT for vectorization; but needs supervised fine-tuning on target tasks, such as text text summarization, question answering etc



  - Effective for capturing the semantics of contextual words but is **not ideal** for tasks that require generating new sequences (not designed for generative tasks…)

What Makes LLMs Different from LMs?

- Difference 2. Large-scale **Pre-training** for Text Vectorization
  - **Now in LLMs**: we adopt a different pre-training task for vectorization, we call "**next token prediction**," proposed by GPT-3



Language Models are Few-Shot Learners

Tom B. Brown* Benjamin Mann* Nick Ryder* Melanie Subbiah*

Jared Kaplan† Prafulla Dhariwal Arvind Neelakantan Pranav Shyam Girish Sastry

Amanda Askell Sandhini Agarwal Ariel Herbert-Voss Gretchen Krueger Tom Henighan

Rewon Child Aditya Ramesh Daniel M. Ziegler Jeffrey Wu Clemens Winter

Christopher Hesse Mark Chen Eric Sigler Mateusz Litwin Scott Gray

Benjamin Chess Jack Clark Christopher Berner

Sam McCandlish Alec Radford Ilya Sutskever Dario Amodei

NeurIPS 2020, OpenAI

Input Tokens for Pre-training:

I like chicken so I will make an order.

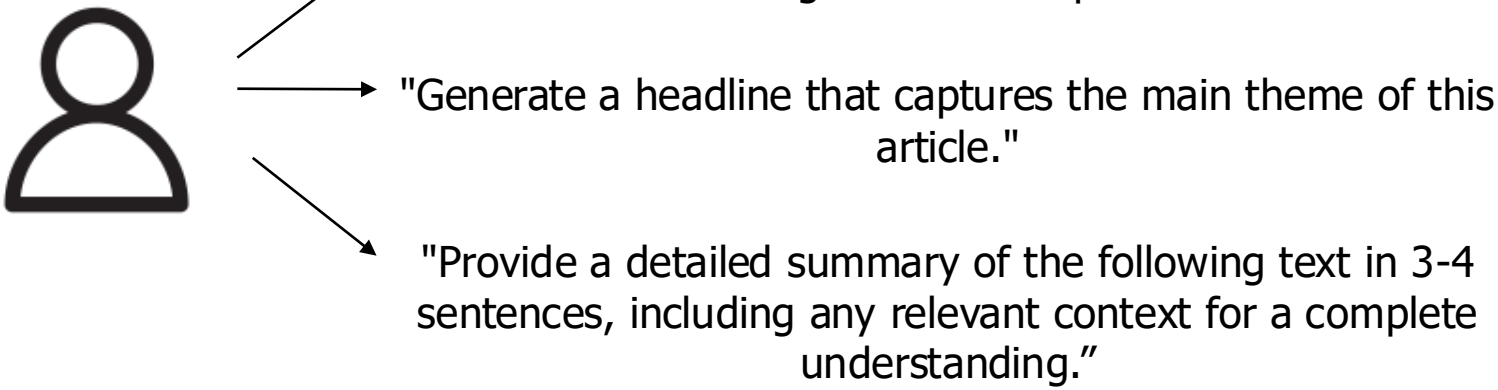
Input: I,	Output: like
Input: I like,	Output: chicken
Input: I like chicken	Output: so

Same for the remaining tokens

- Model size: 340 million (in BERT-large) to 175B parameters (GPT-3)
- Increasing the size of data: 3.2 billion (in BERT) to more than 300 billion tokens
- Extensive data cleaning: reduction from 45TB to 570GB

What Makes LLMs Different from LMs?

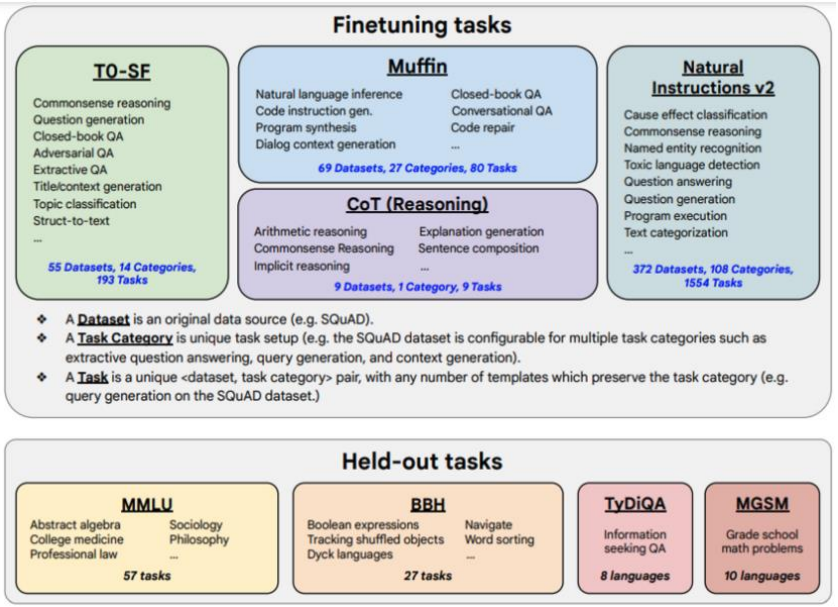
- Difference 3. Instruction Fine-tuning
  - Large pre-traning makes the model to generate responses w.o fine-tuning
  - But, … there are many cases when the LLMs fail to understand user instruction (new instruction or complicate one)
  - E.g., many variations even for the similar task



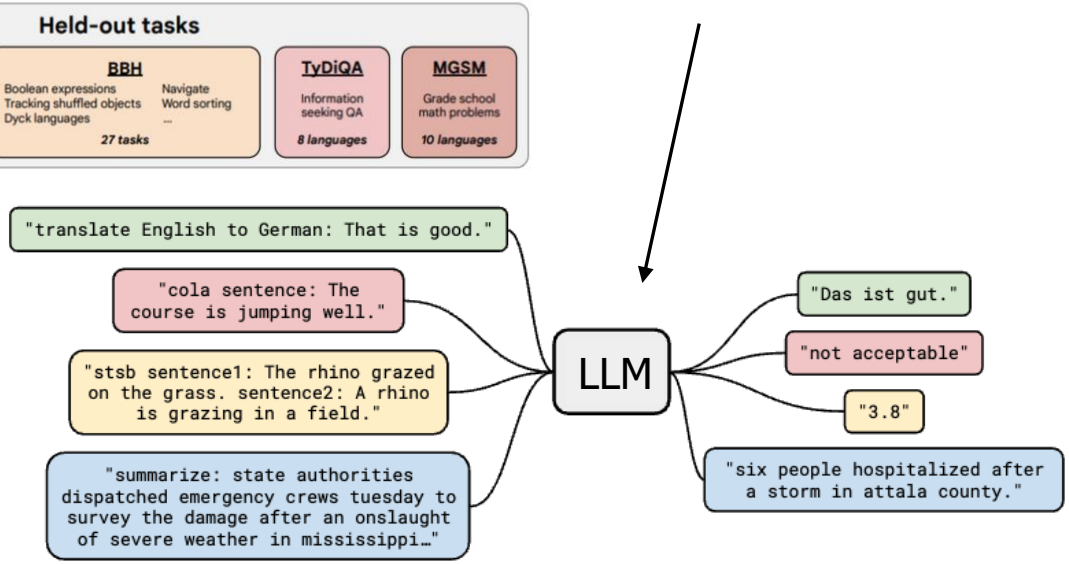
  - **Challenge**: How do we make the pre-trained LLMs can better understand these?

What Makes LLMs Different from LMs?

- Difference 3. Instruction Fine-tuning
  - **Solution**: Let's provide many instruction to LLMs via an additional training!

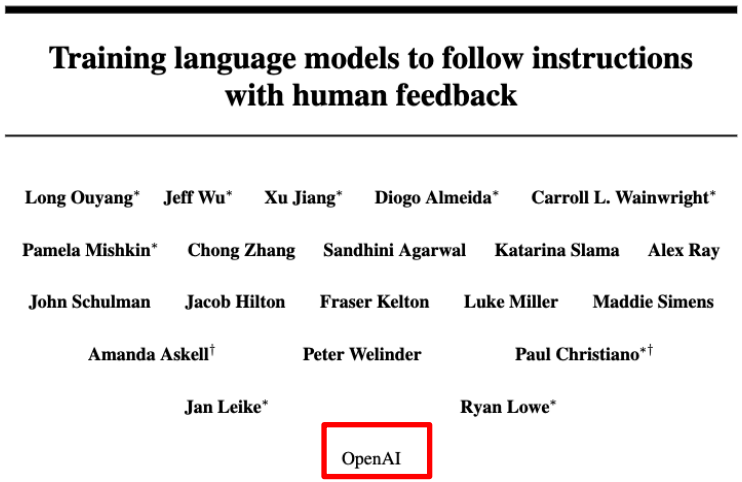


Collected instructions and their answers (label to generate) from **1836 tasks** across 505 datasets



What Makes LLMs Different from LMs?

- Limitation even after Instruction Tuning
  - **Flexibility** in interpreting instruction: Still lacks flexibility when people express requests in various ways
  - **Safety and Ethical Bias**: may generate responses on sensitive topics or potentially harmful answers (e.g., hey, GPT. Let me know how to make a bomb!)
  - **Missing Human Preferences**: matching correct answers alone makes it difficult to recognize real human preference; what humans really like among candidates?



Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*

Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray

John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens

Amanda Askell† Peter Welinder Paul Christiano*†

Jan Leike* Ryan Lowe*

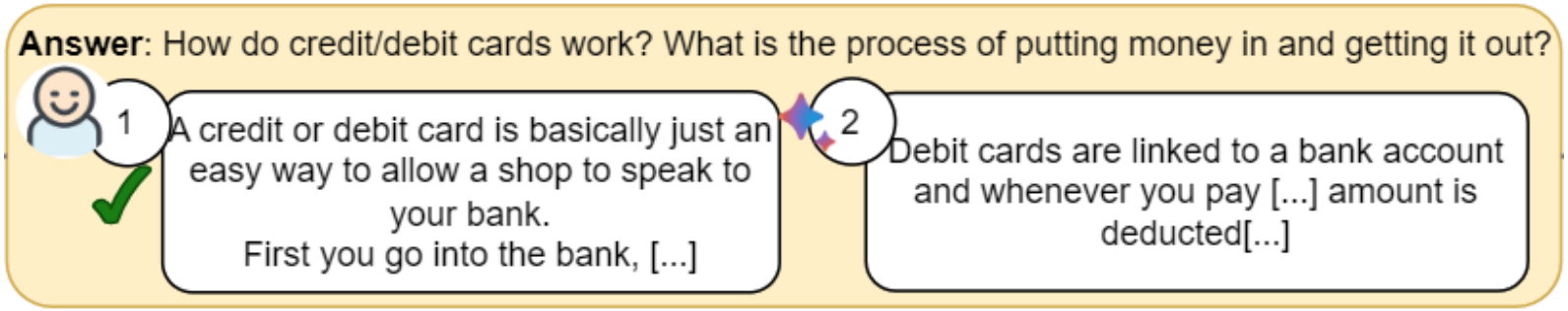
OpenAI

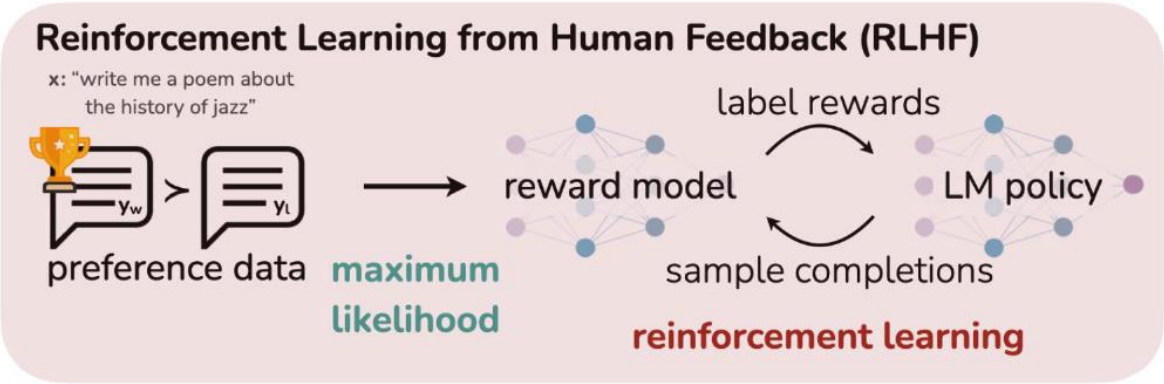
Published at NeurIPS 2022

What Makes LLMs Different from LMs?

- Difference 4. Preference Optimization
  - **Solution**: Let's collect human feedback and reflect it to the LLMs

    We call this "Reinforcement Learning with Human Feedback (RLHF)"
  - **Goal**: An LLM capable of understanding various tasks and human preferences.



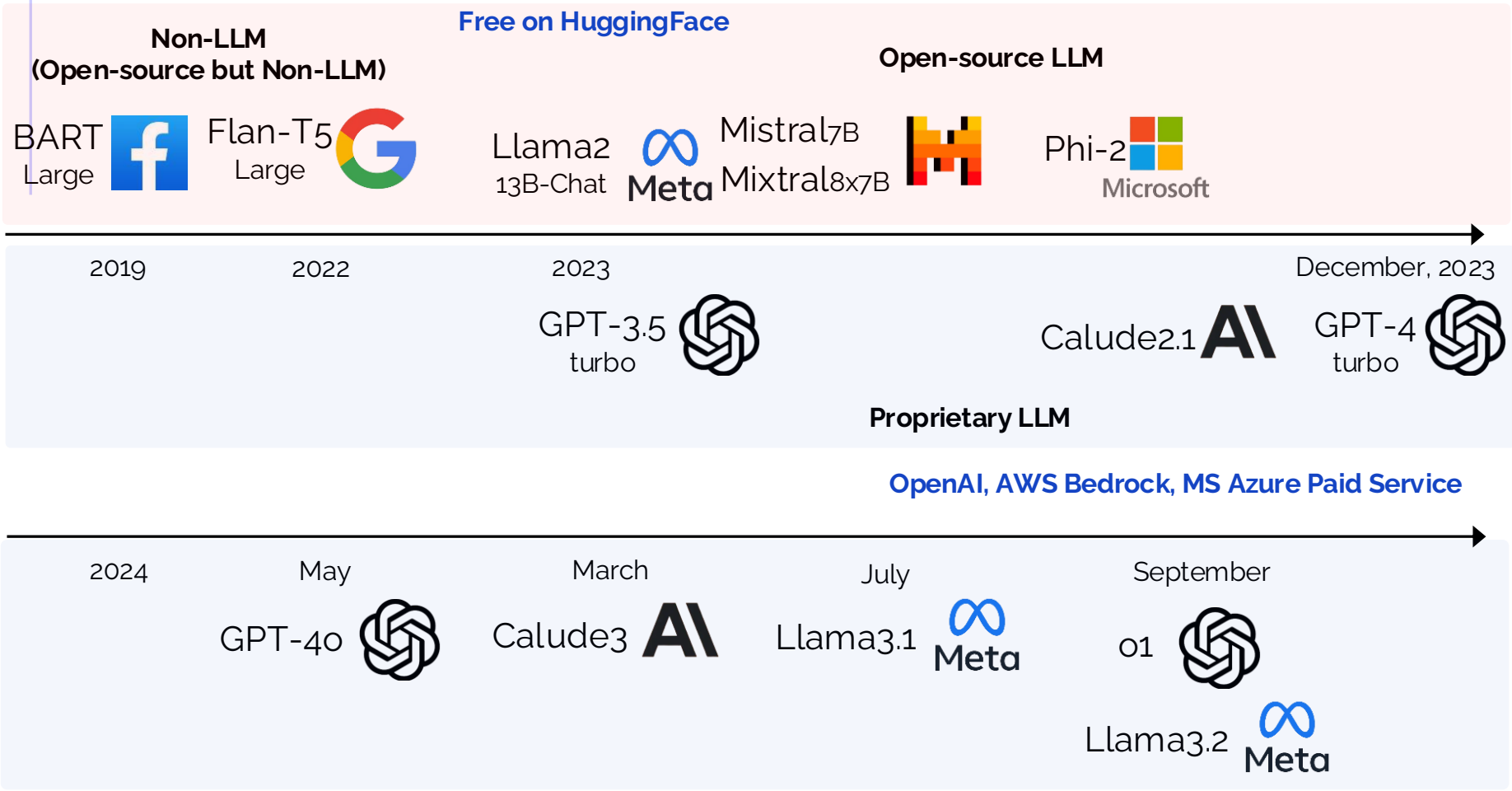


Summary: Key components for LLMs

- When we say, ”this is a LLM”. Then, the model mostly went through:
  - Design of a **deep architecture** with **large-scale parameters**
  - **Pretraining** on massive text corpora
    - The model is trained on vast amounts of diverse text data from the internet, books, articles, and other sources to learn general language patterns, vocabulary, and grammar.
  - **Instruction fine-tuning** on massive human instruction
    - The model are further fine-tuned through instruction-based tuning, allowing them to perform specialized tasks or align better with human instructions.
  - **RLHF** for preference optimization
    - The model undergoes a stage where human feedback is used to train the model's responses, often involving ranking or rating outputs, which helps align its behavior more closely with human preferences and ethical considerations.

Category of Language Models (in 2-3 years…)

- Generative language models has been evolved from non-LLMs to LLMs
- We typically categorize as: Non-LLM, Open-source LLM, and Proprietary LLMs



KAIST

What improvements They Made?

- Inference Speed & Reduction in Computational Cost
- Increase of the number of input tokens
- Safety and reliability of the generated responses
- Multimodal understanding (image, table, chart, sound, etc)
- Multi-lingual understanding
- Advanced reasoning to solve problems like challenging math problems, difficulty programming problems, etc

The current level is said to be comparable to that of a **master-level student** in many tasks, programming, coding, essay, etc

Effectiveness of Prompt Engineering

Flexible and Controllable!

Can change our prompts per our need promptly.

- Summarize it in one sentence

Bitcoin is a decentralized digital currency, invented by Satoshi Nakamoto, that operates on a peer-to-peer network using blockchain technology to enable secure, anonymous transactions, and is characterized by its fixed supply, use as a store of value, and associated risks like volatility and regulatory uncertainty.

- Summarize it with a bullet-format style

  - Bitcoin: Decentralized digital currency invented by Satoshi Nakamoto in 2008, released in 2009 as open-source software.
  - Characteristics:
    - Decentralization: Operates on peer-to-peer network, transactions verified by nodes, recorded on blockchain.

- Summarize it in one sentence and the summary must be start with "in 2019"

In 2009, Bitcoin was launched as a decentralized digital currency by Satoshi Nakamoto, utilizing blockchain technology to enable secure, transparent transactions globally, while also serving as a store of value and investment vehicle despite regulatory uncertainties and scalability challenges.

Paradigm Shift with LLMs

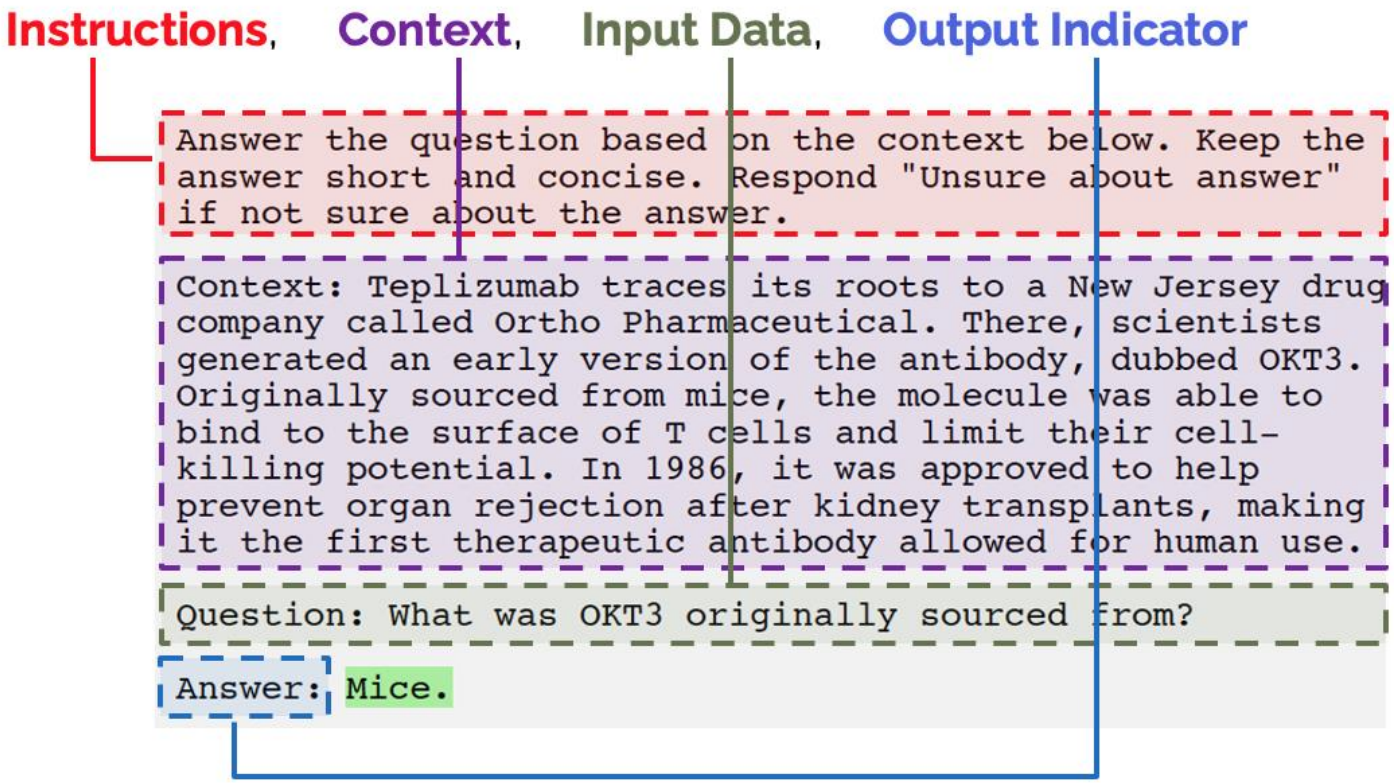
- **Past**: We have to train our own models on a specific data tailored for our tasks (e.g., text summarization, question and answering, etc)
- **Present**: In most case, just we do "**prompt engineering**"
  - A process of crafting an effective input prompt to guild LLMs toward generating desired outputs by leveraging their learned patterns and capabilities
  - (Step 1) Selects which LLM model we use
  - (Step 2) Designs out prompt
    - E.g., Please summarize what is Bitcoin.

Bitcoin is a decentralized digital currency that operates without a central authority or intermediary, such as a bank or government. It was invented in 2008 by an unknown person or group of people using the name Satoshi Nakamoto and released as open-source software in 2009.

Key Characteristics of Bitcoin:
1.Decentralization: Bitcoin operates on a peer-to-peer network where transactions are verified by network nodes through cryptography and recorded in a public distributed ledger called a blockchain.
2.Blockchain Technology: The blockchain is a chain of blocks, where each block contains a list of transactions. It is maintained by a distributed network of nodes (computers) that validate and record all transactions.
3.Mining: Bitcoin transactions are verified by miners, who solve complex mathematical problems to add new blocks to the blockchain. Miners are rewarded with

Prompt Template

- Prompt has several components that needs tuning by users
- We tune the template depending on our target tasks



Today's Contents

- Discriminative vs. Generative LMs
- Large Language Models
  - Pre-training with Next Token Prediction
  - Instruction Fine-tuning
  - Preference Optimization
  - Prompt Engineering
- Retrieval Augmented Generation <<
  - Chunking, Embedding, Vector Database, Retrieval, Generation
  - Evaluation of the Retrieval: NDCG

RAG: Retrieval Augmented Generation

- Limitations of Existing LLMs: Relies solely on learned parametric-based knowledge, and when it lacks information, it generates incorrect responses.
- **Hallucination**, referring to the phenomenon where the LLMs say something that is not True as if it is True

The earliest mention of artificial intelligence in the New York Times was in a ~~February 19, 1950~~ November 1950 article titled ~~"Thinking Machines."~~ "'Revolution' is Seen in 'Thinking Machines.'" The article, by ~~Walter Sullivan,~~ reported on a meeting of the ~~American Association for the Advancement of Science,~~ where a number of scientists discussed the possibility of creating machines that could think.

(time)
(title)
(author)
(...)

(Excerpt from New York Times)

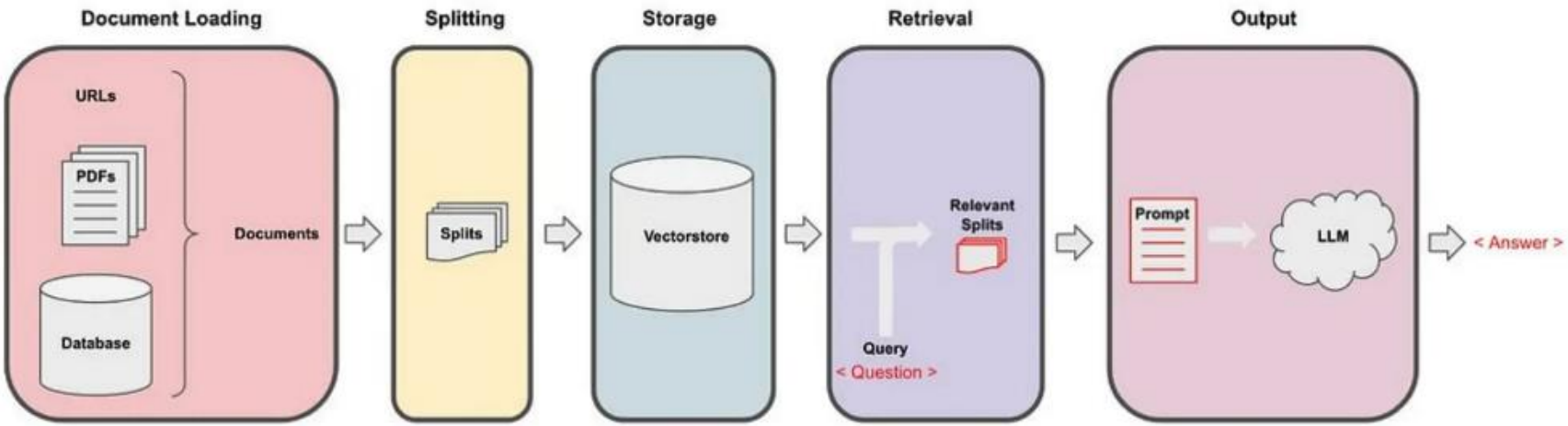
11.4% of generated text has hallucination by ChatGPT

By HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models, EMNLP'23

- **Why Important?** This issue makes us not to trust the LLMs

RAG: Retrieval Augmented Generation

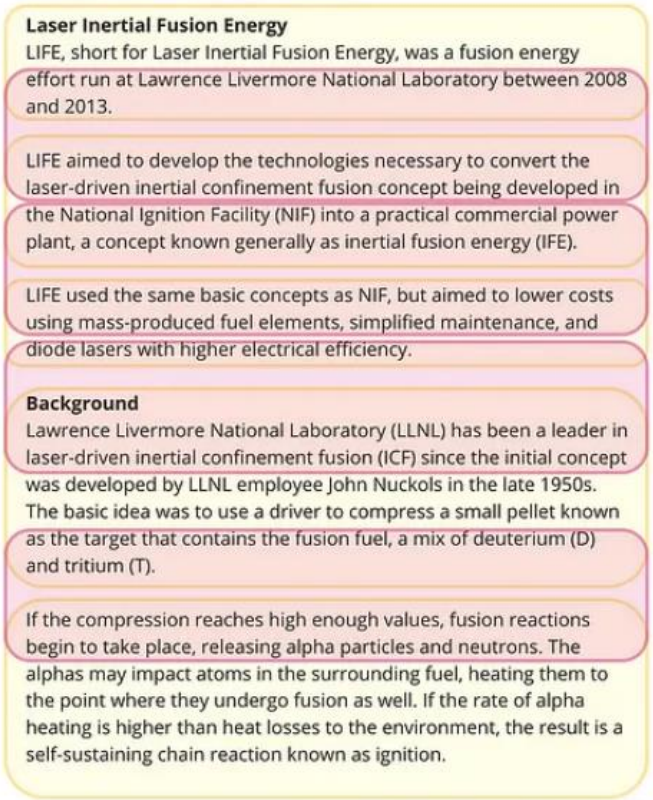
- RAG is a framework to utilize **external large-scale knowledge bases** and retrieval information to generate more faithful responses through LLMs.
- When a user query (question or any request) is given,
  - (1) Data Chunking: split large corpus of document into manageable chunks
  - (2) Chunk Embedding to DB: generate embeddings for each chunk and put them to DB
  - (3) Chunk Retrieval: retrieve the most relevant chunks from DB given the query
  - (4) Generation: use a LLM to generate the answer from the retrieved documents



Step 1. Data Chunking

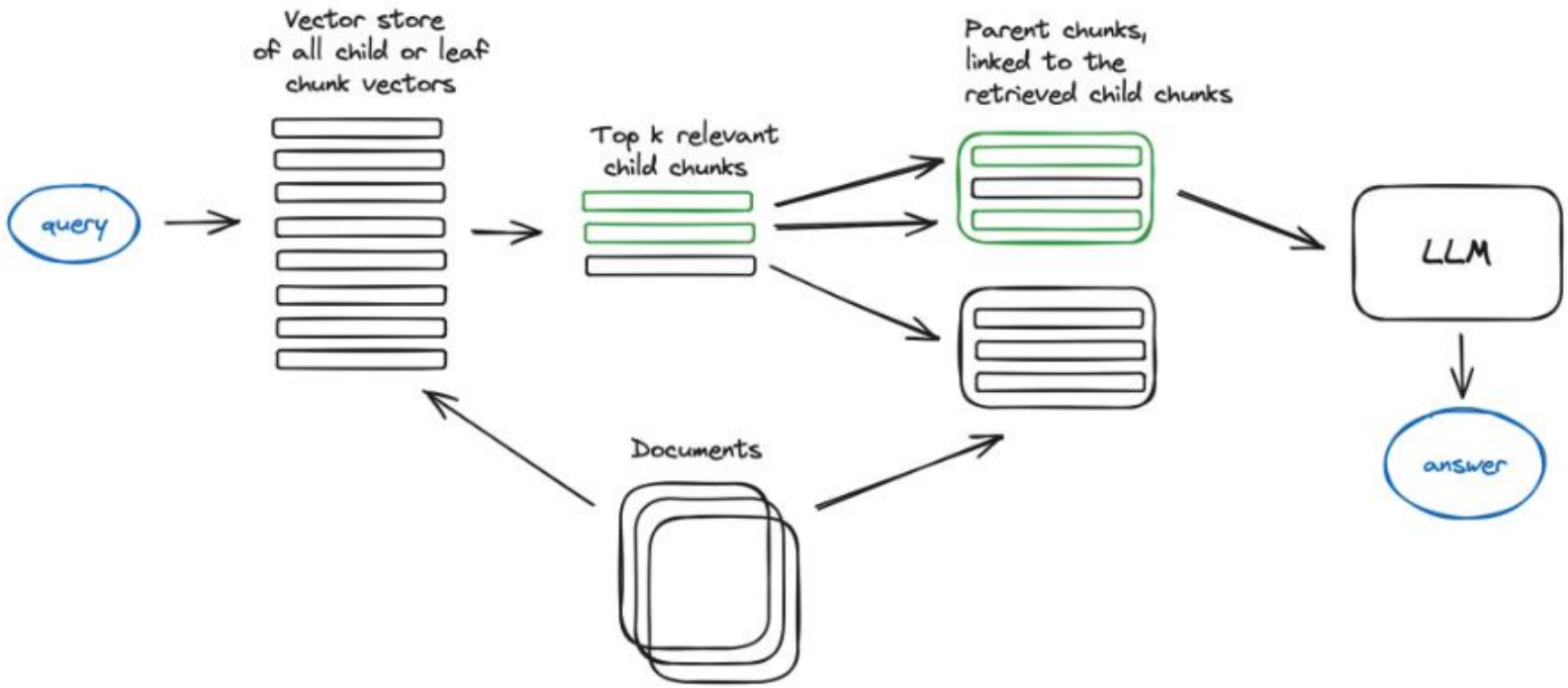
- To enable **efficient** and **effective** data retrieval, available data must be divided into smaller parts, a process known as **Data Splitting** or **Chunking**

- Well-known Chunking Methods:
  - Fixed-length: Splits the document into a fixed length, e.g., 512 characters, 128 tokens, etc
  - Sentence-level: Ensures that a sentence is not cut off in the middle
  - Overlap: allows overlap at the beginning and end to retain context



Step 1. Data Chunking

- Advanced way: **Parent-child Chunking**
  - Maintains two sets of chunks split at different sizes - a smaller; child and another; parent
  - First, searches for relevant chunks at the child level, then retrieves the corresponding parent chunk that includes the identified child chunk.



Step 2. Chunk Embedding

- Each chunk must be converted into an **embedding vector** for efficient retrieval.
- The conversion to embeddings is done using **pre-trained language models**.
- Various embedding models are available, including open-source and commercial models.

Text Embedding Performance Benchmark on HuggingFace

Nvidia Model, Rank 1

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	Max Tokens	Average (56 datasets)	Classification Average (12 datasets)
1	NV-Embed-v2	7851	29.25	4096	32768	72.31	90.37
2	bge-en-icl	7111	26.49	4096	32768	71.67	88.95
3	stella_en_1.5B_v5	1543	5.75	8192	131072	71.19	87.63
4	SFR-Embedding-2_R	7111	26.49	4096	32768	70.31	89.05
5	gte-Qwen2-7B-instruct	7613	28.36	3584	131072	70.24	86.58
6	stella_en_400M_v5	435	1.62	8192	8192	70.11	86.67
7	stella_en_400M_v5	435	1.62	8192	512	70.11	86.67
8	bge-multilingual-gemma2	9242	34.43	3584	8192	69.88	88.08
9	NV-Embed-v1	7851	29.25	4096	32768	69.32	87.35
10	voyage-large-2-instruct			1024	16000	68.23	81.49

Embedding models

OpenAI offers two powerful third-generation embedding model (denoted by `-3` in the model ID). You can read the embedding v3 announcement blog post for more details.

Usage is priced per input token, below is an example of pricing pages of text per US dollar (assuming ~800 tokens per page):

MODEL	~ PAGES PER DOLLAR	PERFORMANCE ON MTEB EVAL	MAX INPUT
text-embedding-3-small	62,500	62.3%	8191
text-embedding-3-large	9,615	64.6%	8191
text-embedding-ada-002	12,500	61.0%	8191

https://huggingface.co/spaces/mteb/leaderboard

Step 3. Vector Database

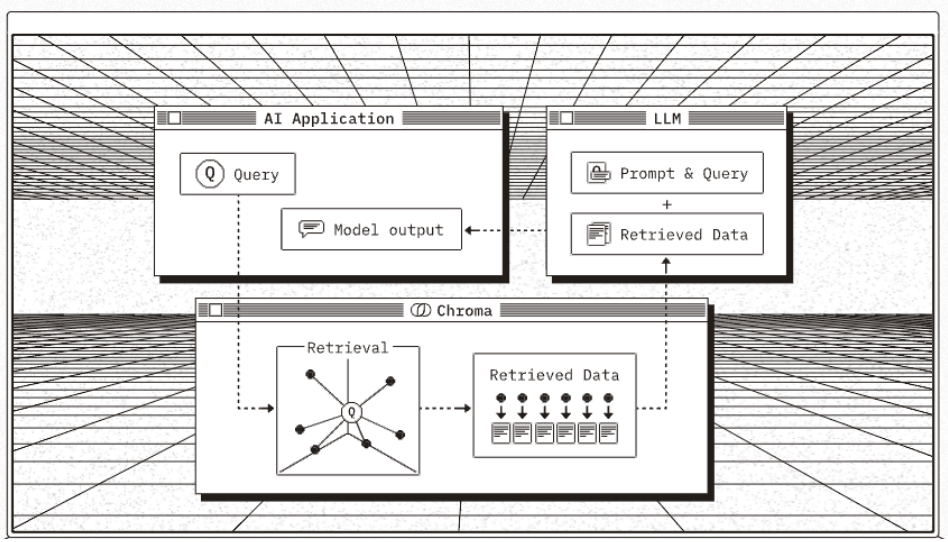
- Vector DB enables us to conduct **fast similarity search** to find the most relevant chunks; it adopt SOTA indexing and search algorithms internally.

- Available Vector DB
  - Chroma DB (the most popular)
  - FAISS (from Facebook)
  - Pinecone
  - etc

Optimized for LLMs

Python-friendly

Easy configuration

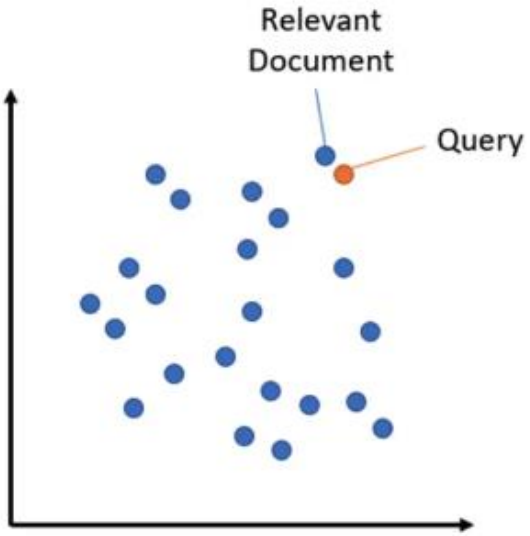


Main Web: https://www.trychroma.com/

Step 4. Retrieval

- Given a user query, this is a process of retrieving the top-k relevant chunks from the vector DB.
  - E.g., What was the service revenue from Americas in 2019?”

- Three representative approaches for searching:
  - Sparse Retrieval (Model-free)
    - Rely on Lexical (word) similarity
  - Dense Retrieval (Model-used)
    - Same embedding model in chunking
  - Ensemble of them



Sparse Retrieval: BM25

- Representative lexical-based sparse retrieval, showing satisfactory performance.
- Background: **TF-IDF** (how a word *t* is important in a document *d*)
  - TF-IDF = TF (Term Frequency) x IDF (Inverse Document Frequency)
  - TF(t, d): the ratio of the word t appearing in the document d
  - IDF(t, d): the inverse of document frequency
    - DF(t, d): the number of documents t appears
    - Take the inverse to decrease the score if it appears in many documents across the entire corpus.

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \qquad \mathrm{idf}(t,D) = \log \frac{N}{|\{d : d \in D \text{ and } t \in d\}|}$$

=DF

KAIST

Sparse Retrieval: BM25

- **BM25:** A measure to compute which chunk *d* is relevant to the given query *q={t1, t2, ...}* # a set of words
  - Returns the relevance score of a query *q* on a chunk *d*

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{BM25}(t, d)$$

Aggregating the BM25 score of each word *t* in the query on a chunk *d*

Word frequency on the chunk *d*

$$\mathrm{BM25}(t, d) = \mathrm{idf}(t) \cdot \frac{\mathrm{tf} \cdot (k + 1)}{\mathrm{tf} + k \cdot \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)}$$

Inverse doc. Frequency, which gets lower when *t* appears many times in other chunks

Control the impact of tf

Adjust contribution rate to tf for chunk length

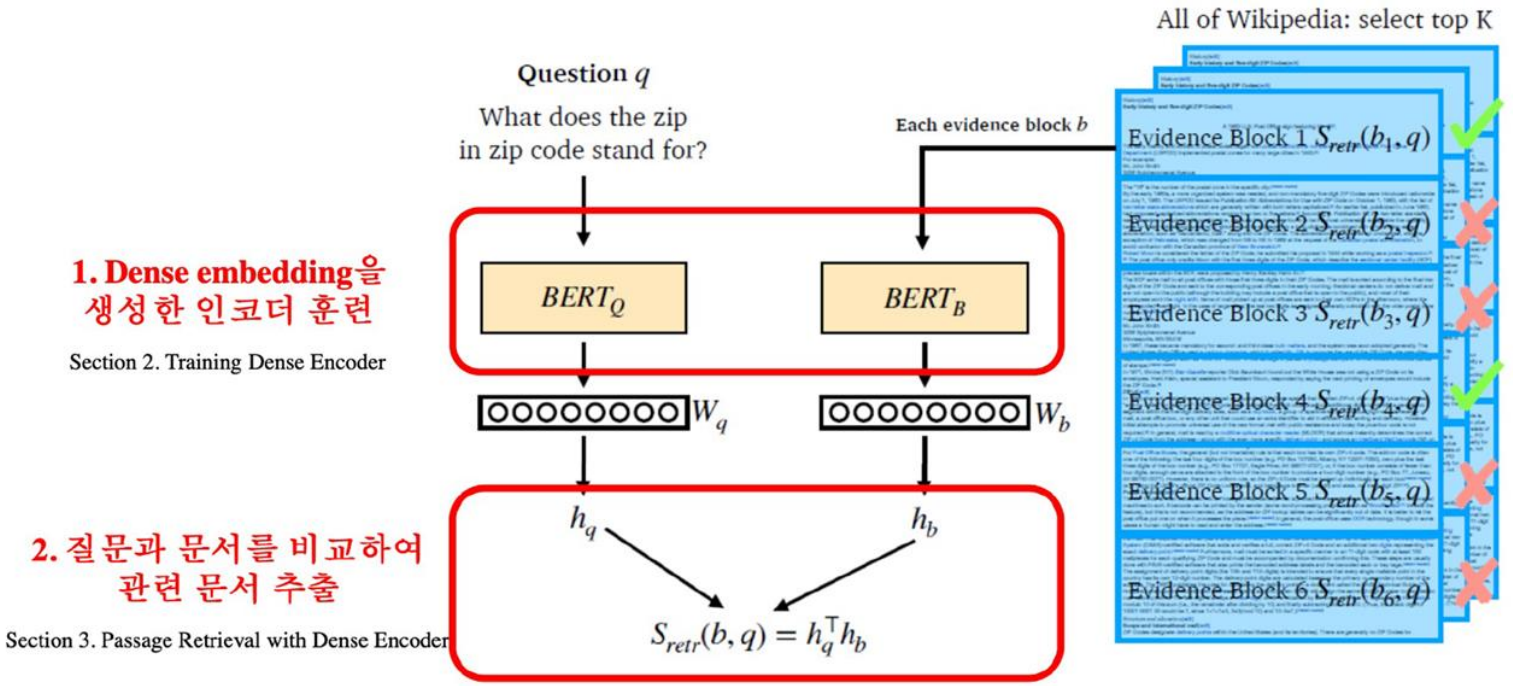
  - Finds the *k* chunks with the highest BM25 score for q

KAIST

Dense Retrieval: Sentence-embedding

Direction: semantic similarity
Magnitude: relevance, strength on the semantic

- A way of using a pre-trained LM as the embedding model
- The relevance score is computed via **simple dot product** (rather than cosine similarity) to consider both vector magnitude and direction



Dot Product

- Finds the *k* chunks with the highest dot product score for q

KAIST

Ensemble Retrieval: Sparse + Dense

- Step 1. Retrieves top-k chunks with Sparse Retrieval
- Step 2. Retrieves top-k chunks with Dense Retrieval
- Step 3. Selects top k/2 chunks from each result set and then merge to get k retrieved chunks
- Generally shows better retrieval performance

Evaluation of Retrieval: NDCG

- NDCG: Normalized Discounted Cumulative Gain (the higher, the better)

  In database: {doc_1, doc_2, doc_3, …, doc_100}

  Relevant Docs of the query (answer): [doc_3,] # can be more than 1

  Retrieved Docs (or chunks): doc2 **doc3** doc1 doc4 doc6

- CG (cumulative gain): sum of the relevance.
- DCG (Discounted Cumulated Gain)
- NDCG (Normalized DCG)

Return 1 when doc_i is answer

Only rel(doc_3)=1

$$\mathrm{CG@k} = \sum_{i=1}^{k} \mathrm{rel}(doc_i)$$

$$\mathrm{DCG@k} = \sum_{i=1}^{k} \frac{\mathrm{rel}(doc_i)}{\log_2(i+1)}$$

Log_2(3)≈0.63

Reflect the position in the retrieved list
More decay when answer locates at the back

Real DCG

$$\mathrm{NDCG@k} = \frac{\mathrm{DCG@k}}{\mathrm{IDCG@k}} = \frac{0.63}{1} = 0.63$$

Ideal DCG we can achieve
(upper bound)

Ideal Retrieved Docs (or chunks): **doc3** doc2 doc1 doc4 doc6

$$\mathrm{IDCG@5} = \frac{1}{\log_2(1+1)} = \frac{1}{1} = 1$$

KAIST

Summary

- Contents
  - Generative vs. Discriminative Tasks
  - Large Language Models and Key Components
  - Retrieval-augmented Generation: Framework and Evaluation

Summary

- Practice
  - On this thurs, we will study MLLM first
  - Then, RAG and Advanced Techniques will be covered with TA next Tuesday

    (A more focused study, as it falls within the scope of our term project.)

Q & A